

Java Weather App: Full Development Plan

A beginner-to-intermediate Java project, built the way real software actually gets built: scoped, layered, tested, and version-controlled.

0. What you're building

MVP (Minimum Viable Product): A console app. User types a city name, the app fetches current weather from the OpenWeather API and prints temperature, conditions, humidity, and wind speed. It handles bad input and network failures without crashing.

Why this project is worth your time: It's the smallest project that forces you to touch HTTP, JSON, external configuration, custom exceptions, layered architecture, and mockable unit testing, all in one place. That's the skill cluster that separates "I can write Java syntax" from "I can build a Java application."

Explicitly out of scope for v1: GUI, database, forecasts, maps, multi-threading. These live in the stretch goals in Section 9. Resist scope creep. Finishing v1 matters more than starting v3.

1. Prerequisites & environment setup

1.1 Required knowledge before starting

You should already be comfortable with:

- Variables, loops, conditionals, methods
- Classes, objects, constructors, getters
- `ArrayList` / `HashMap` basics
- `try/catch` basics

If any of these feel shaky, do Section 10.1 first. Don't start this project on a weak OOP foundation, you'll end up copying code you don't understand.

1.2 Tooling

Tool	Version	Why
JDK	17 or 21 (LTS)	<code>java.net.http.HttpClient</code> needs Java 11+. Use an LTS release.
IntelliJ IDEA Community	Latest	Free, best-in-class Java IDE. VS Code + Extension Pack for Java also works fine.
Maven	3.9+	Dependency management and build. Bundled with IntelliJ.
Git	Latest	Version control from commit #1, not bolted on at the end.
Postman or browser	n/a	Inspect raw API responses before writing any Java.

1.3 Verify your setup

```
java -version      # should print 17.x or 21.x
mvn -version
git --version
```

1.4 Get your API key

1. Sign up free at OpenWeather.
 2. Verify your email.
 3. Copy your key from the API keys tab.
 4. **Wait.** New keys can take up to about 2 hours to activate. An immediate `401 Invalid API key` is expected and isn't a bug in your code. This trips up almost everyone.
-

2. The API: understand it before you code

2.1 Two endpoints, two steps

Step 1, Geocoding API (city name to coordinates):

```
https://api.openweathermap.org/geo/1.0/direct?q={city},{countryCode}&limit=5&appid={KEY}
```

Returns an array of matching locations with `lat`, `lon`, `name`, `country`, `state`.

Step 2, Current Weather API (coordinates to weather):

```
https://api.openweathermap.org/data/2.5/weather?lat={lat}&lon={lon}&appid={KEY}&units=metric
```

`lat`, `lon`, `appid` are mandatory. `units` and `lang` are optional. Set `units=metric` if you want Celsius and m/s, otherwise you'll get Kelvin and wonder why the temperature says 301.

2.2 Why two calls instead of one

The single-call `?q=CityName` shortcut still works but is deprecated. Doing it properly gets you a few things:

- It teaches you to chain dependent API calls, which comes up constantly in real work
- It handles ambiguity correctly. "Springfield" matches many places, so geocoding returns all of them and lets the user pick
- It future-proofs your app against the shortcut eventually being removed

2.3 Response shape to plan for

The weather response nests data across several objects: a `main` object (temperature, feels-like, humidity, pressure), a `weather` **array** (condition id, main, description, icon), a `wind` object (speed, degrees), plus `name`, `dt`, `sys.country`, `sys.sunrise`, `sys.sunset`.

Watch out for `weather`. It's an array, and you almost always want `weather[0]`. Beginners routinely try to read it as an object and get a parse error.

2.4 Do this before writing Java

Paste a real request URL into your browser. Read the raw JSON. Map every field you care about onto a variable name on paper. Then open your IDE.

3. Architecture

3.1 Design principle: layered separation

```
+-----+
| Presentation Layer (ConsoleUI) | talks to the human
+-----+
| Service Layer (WeatherService) | orchestrates, business logic
+-----+
| Client Layer (ApiClient) | raw HTTP only
+-----+
| Model Layer (Weather, Location) | plain data
+-----+
Config (AppConfig) sits alongside all of it
```

The rule that makes this work: each layer only knows about the layer directly below it. `ConsoleUI` never sees JSON. `ApiClient` never sees a `Weather` object. That one constraint is what makes the whole thing testable.

3.2 Package structure

```
src/main/java/com/princeblue/weather/
├─ Main.java
├─ config/
│   └─ AppConfig.java
├─ ui/
│   └─ ConsoleUI.java
├─ service/
│   ├── WeatherService.java
│   └─ WeatherMapper.java
├─ client/
│   ├── ApiClient.java
│   └─ OpenWeatherClient.java (implements ApiClient)
├─ model/
│   ├── Weather.java
│   └─ Location.java
└─ exception/
    ├── WeatherAppException.java (base)
    ├── CityNotFoundException.java
    ├── ApiUnavailableException.java
    └─ InvalidApiKeyException.java

src/test/java/com/princeblue/weather/
├─ service/WeatherServiceTest.java
├─ service/WeatherMapperTest.java
└─ client/OpenWeatherClientTest.java

src/main/resources/
└─ config.properties (gitignored)
```

3.3 Class responsibilities

Main : entry point only. Wires the dependencies together, hands control to `ConsoleUI`. Should be under 20 lines.

AppConfig : loads the API key and base URLs from `config.properties` or an environment variable. Fails loudly at startup if the key is missing. Never hardcode the key.

ConsoleUI : the input loop. Prompts, reads with `Scanner` , calls the service, formats output, catches `WeatherAppException` and prints a friendly message. Contains zero HTTP and zero JSON.

ApiClient (interface): declares `String get(String url)` . This interface is the seam that lets you mock the network in tests. Defining it is probably the single most important architectural decision in the project.

OpenWeatherClient : the real implementation. Builds the `HttpRequest` , sends it via `HttpClient` , checks the status code, returns the raw response body as a `String` . Translates HTTP status codes into your custom exceptions (401 to `InvalidApiKeyException` , 404 to `CityNotFoundException` , 5xx to `ApiUnavailableException`).

WeatherMapper : a pure function. Raw JSON `String` in, `Weather/Location` object out. No network, no I/O. Easy to unit-test with saved JSON fixtures.

WeatherService : orchestration. `getWeather(cityName)` geocodes, picks a location, fetches weather, maps it, returns it. This is where the two-step flow lives.

Weather / Location : immutable POJOs. Final fields, constructor, getters, `toString()` . No logic.

Exceptions: a base `WeatherAppException` with specific subclasses lets `ConsoleUI` catch one type but still print a tailored message.

3.4 Sequence for one lookup

```
User types "Pune"
-> ConsoleUI.run()
  -> WeatherService.getWeather("Pune")
    -> ApiClient.get(geocodeUrl)           [HTTP #1]
    -> WeatherMapper.toLocations(json)      -> List<Location>
    -> (if multiple, ConsoleUI disambiguates)
    -> ApiClient.get(weatherUrl(lat,lon))   [HTTP #2]
    -> WeatherMapper.toWeather(json)       -> Weather
  -> ConsoleUI formats and prints
```

3.5 Dependencies (pom.xml)

```
<dependencies>
  <dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.11.0</version>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>5.11.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Check Maven Central for newer versions when you actually set this up. Gson over Jackson here, mostly because it's a simpler API and a smaller learning surface for a first project.

4. Development roadmap

Each phase should end with a working, committed, runnable state. Don't leave the repo broken overnight.

Phase 0, Repo setup (about 30 min)

- `git init`, create a GitHub repo
- `.gitignore`: `target/`, `*.iml`, `.idea/`, `config.properties`, `.env`
- Maven project skeleton, `pom.xml` with dependencies
- `README.md` stub
- **Commit:** `chore: initial project scaffold`

Phase 1, Raw connectivity (1 to 2 hrs)

Goal: prove you can hit the API from Java.

- Hardcode a URL in `Main`
- `HttpClient` to `HttpRequest` to `send()` to `System.out.println(response.body())`
- **Done when:** raw JSON prints to your console
- **Commit:** `feat: raw HTTP call to weather endpoint`

This is the phase where most people get stuck on the inactive API key. Test the same URL in your browser first. If the browser fails too, it's the key, not your code.

Phase 2, Configuration (about 45 min)

- Create `config.properties` with `api.key=...`
- `AppConfig` loads it, throws a clear error if it's absent
- Add `config.properties.example` (with a placeholder) to Git so others know the format
- **Done when:** no secret exists anywhere in your Git history
- **Commit:** `feat: externalize API key configuration`

Phase 3, Models & mapping (2 to 3 hrs)

- Write `Weather` and `Location` as immutable POJOs
- Save a real API response to `src/test/resources/weather-response.json`
- `WeatherMapper` parses that JSON with `Gson` into your objects
- **Done when:** you can print `weather.toString()` with real values
- **Commit:** `feat: add domain models and JSON mapper`

Phase 4, Layer extraction (about 2 hrs)

- Define the `ApiClient` interface, move HTTP into `OpenWeatherClient`
- Create `WeatherService` to orchestrate geocode then weather
- `Main` now just wires objects together
- **Done when:** behaviour is unchanged, but `Main` is around 15 lines
- **Commit:** `refactor: extract client and service layers`

Nothing new works after this phase, and that's fine. Refactoring is changing structure without changing behaviour, and learning to do it on purpose is a real skill.

Phase 5, Console UI & loop (about 2 hrs)

- `ConsoleUI` with a prompt loop, `quit` to exit
- Clean formatted output (aligned labels, units, condition description)
- Multi-match disambiguation: list options, let the user pick by number
- **Commit:** feat: interactive console interface

Phase 6, Error handling (2 to 3 hrs)

Build the exception hierarchy and handle every row in this table:

Scenario	Trigger	Behaviour
Empty input	User hits Enter	Re-prompt, no API call
City not found	Geocoding returns <code>[]</code>	"No city found matching X. Check spelling."
Invalid API key	HTTP 401	"API key rejected. Check config / wait for activation."
Rate limit	HTTP 429	"Too many requests. Wait a minute."
Server error	HTTP 5xx	"Weather service unavailable. Try later."
No internet	<code>IOException</code> / <code>ConnectException</code>	"Cannot reach the network."
Timeout	Request exceeds 10s	"Request timed out."
Malformed JSON	<code>JsonSyntaxException</code>	"Unexpected response from server."

Set an explicit timeout on your `HttpRequest`. Without one, a hung connection hangs your whole app.

- **Done when:** you can't crash the app with any input, and it survives your Wi-Fi getting switched off
- **Commit:** feat: comprehensive error handling

Phase 7, Tests (3 to 4 hrs)

See Section 5 for the full breakdown.

- **Commit:** test: unit tests for mapper, service, client

Phase 8, Polish & documentation (about 2 hrs)

- Javadoc on every public class and method
- Full `README.md` (Section 7)
- Consistent formatting, remove dead code and stray `println`s
- Tag the release: `git tag v1.0.0`
- **Commit:** docs: project documentation and cleanup

Realistic total: 18 to 25 hours, spread over 2 to 3 weeks at a student pace. If it takes longer, that's normal and not a signal about your ability.

5. Testing strategy

5.1 Why your architecture makes this possible

Because `WeatherService` depends on the `ApiClient` **interface** rather than the concrete HTTP class, you can inject a fake in tests. No network, no API key, no rate limits, instant runs. That's dependency injection, and it's the practical payoff of Section 3.

5.2 Test layers

Unit tests, `WeatherMapperTest` (easiest, start here)

- Load saved JSON fixtures from `src/test/resources/`
- Assert every mapped field
- Cases: valid response, missing optional field, empty `weather` array, malformed JSON, empty geocoding array

Unit tests, `WeatherServiceTest` (with Mockito)

```
@Test
void returnsWeatherForValidCity() {
    ApiClient mockClient = mock(ApiClient.class);
    when(mockClient.get(contains("geo/1.0/direct")))
        .thenReturn(loadFixture("geocode-pune.json"));
    when(mockClient.get(contains("data/2.5/weather")))
        .thenReturn(loadFixture("weather-pune.json"));

    WeatherService service = new WeatherService(mockClient, config);
    Weather result = service.getWeather("Pune");

    assertEquals("Pune", result.getCityName());
    assertEquals(28.5, result.getTemperature(), 0.01);
}

@Test
void throwsWhenCityNotFound() {
    ApiClient mockClient = mock(ApiClient.class);
    when(mockClient.get(anyString())).thenReturn("[]");

    WeatherService service = new WeatherService(mockClient, config);

    assertThrows(CityNotFoundException.class,
        () -> service.getWeather("Xyzzyville"));
}
```

Unit tests, `OpenWeatherClientTest`

- URL construction: does the built URL contain the right params, properly encoded?
- Critical case: a city with a space ("New Delhi") or non-ASCII characters has to be URL-encoded. Test this explicitly, it's a classic silent bug.
- Status-code-to-exception translation

Integration tests (small number, tagged separately)

- One test that hits the real API, tagged `@Tag("integration")`
- Excluded from the default Maven run so your test suite works offline
- Run manually before a release to confirm the API contract hasn't changed

Manual test checklist (run before tagging v1.0)

- ☐ Valid city returns correct data
- ☐ Ambiguous city ("Springfield") triggers the disambiguation list
- ☐ City with a space in the name works
- ☐ Nonsense string gives a friendly error
- ☐ Empty input re-prompts
- ☐ Wi-Fi off gives a friendly error, no stack trace
- ☐ Wrong key in config gives a clear message
- ☐ `quit` exits cleanly
- ☐ 10 lookups in a row, no degradation

5.3 Coverage target

Aim for around 70 to 80% on the service and mapper layers. Don't chase 100%, testing getters is busywork. Use JaCoCo (`jacoco-maven-plugin`) to measure, and read the report for the uncovered branches rather than the headline number.

5.4 Run tests

```
mvn test                # unit only
mvn test -Dgroups="integration" # integration only
mvn verify              # everything + coverage report
```

6. Git workflow

- **Branch per feature:** `feat/error-handling`, `feat/console-ui`
- **Conventional commits:** `feat:`, `fix:`, `refactor:`, `test:`, `docs:`, `chore:`
- **Small commits.** One logical change each. "Did lots of stuff" is not a commit message.
- **PR to main even solo.** Write a description of what changed and why. Reviewing your own diff catches real bugs.
- **Never commit secrets.** If you do, treat the key as compromised: revoke and regenerate it. Removing it in a later commit doesn't remove it from history.

Optional stretch: a GitHub Actions workflow that runs `mvn test` on every push. It's maybe 15 lines of YAML and makes the repo look a lot more finished.

7. README structure

Your README is what a recruiter or maintainer actually reads. Include:

1. **Title + one-line description**
2. **Demo:** a terminal screenshot or asciinema GIF
3. **Features:** bulleted
4. **Tech stack:** Java 17, Maven, Gson, JUnit 5, Mockito, OpenWeather API
5. **Architecture:** the layer diagram from Section 3.1
6. **Setup:** prerequisites, clone, get an API key, copy `config.properties.example`, `mvn clean install`, `mvn exec:java`
7. **Running tests**
8. **Project structure:** the tree
9. **Roadmap:** your stretch goals, checkboxed
10. **License:** MIT is a fine default

8. Common pitfalls

Pitfall	Fix
API key returns 401 immediately	Wait up to 2 hours after signup. Not your code.
Temperatures in the 290s	You forgot <code>units=metric</code> . That's Kelvin.
<code>weather</code> array parse error	It's a JSON array, read <code>weather[0]</code> .
Cities with spaces fail	URL-encode the query parameter.
App hangs forever	Set an explicit request timeout.
Secret committed to Git	Revoke and regenerate the key immediately.
Everything crammed in <code>Main</code>	Do Phase 4. Seriously.
Tests need internet	Mock <code>ApiClient</code> . That's what the interface is for.

9. Stretch goals (in difficulty order)

1. **5-day forecast.** New endpoint, grouped display. Low effort, high visual payoff.
2. **Search history + favourites.** Persist to a JSON file. Teaches file I/O.
3. **Response caching.** Cache for 10 minutes to avoid hammering the API. Teaches TTL logic.
4. **Unit toggle.** Celsius/Fahrenheit switch at runtime.
5. **ASCII art conditions.** Map condition codes to ASCII sun/cloud/rain. Pure fun, and surprisingly memorable in a demo.
6. **JavaFX GUI.** Search box, results card, condition icons. Your layered architecture means you only swap the UI layer and everything else stays untouched. That's the moment the architecture lesson actually lands.
7. **Geolocation default.** Detect approximate location via IP on startup.
8. **Spring Boot REST wrapper.** Expose your own `/api/weather?city=X`. A natural bridge into backend development.

Do 1 through 3 before touching 6. A polished console app beats a half-finished GUI.

10. Learning resources

Organized by what you'll need, roughly in the order you'll need it. Official docs work well as reference, video is better for first exposure.

10.1 Java & OOP fundamentals (only if needed)

- **Official Java Tutorials (Oracle):** <https://docs.oracle.com/javase/tutorial/>. Dry but authoritative; the "Learning the Java Language" trail covers OOP properly.
- **Java SE 21 API Docs:** <https://docs.oracle.com/en/java/javase/21/docs/api/>. Bookmark this. Learning to read Javadoc is a skill in itself.
- **Baeldung:** <https://www.baeldung.com/>. Probably the best practical Java site around, free articles on nearly everything here.
- **YouTube:** Bro Code's Java full course (around 12 hours, hundreds of code examples and quizzes, good for a linear first pass if you're a visual learner). Telusko (Navin Reddy) is strong specifically on OOP, interfaces, collections, and exception handling. Search `Bro Code Java Full Course` or `Telusko Java Tutorial for Beginners`.

A note on method: pick one channel per stage and code along with it. The common mistake is watching several tutorials passively without writing anything. You already know this from learning Python, same rule applies here.

10.2 HTTP & REST concepts

- **MDN, HTTP overview:** <https://developer.mozilla.org/en-US/docs/Web/HTTP>. Probably the best HTTP reference anywhere.
- **MDN, HTTP status codes:** <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>. You'll need 200, 401, 404, 429, 500.
- **What is a REST API:** search for freeCodeCamp's REST API crash course.
- Concepts to actually understand: request/response, GET vs POST, headers, query parameters, status codes, JSON as a transport format.

10.3 Java HttpClient (Java 11+)

- **Baeldung, Java HttpClient guide:** <https://www.baeldung.com/java-9-http-client>. The modern API, introduced as an incubator in Java 9 and standardized in Java 11, so it needs no extra dependencies.
- **Baeldung, simple HTTP requests:** <https://www.baeldung.com/java-http-request>. Includes the older `HttpURLConnection` for contrast.
- **Official java.net.http docs:** <https://docs.oracle.com/en/java/javase/21/docs/api/java.net.http/java/net/http/package-summary.html>
- **YouTube search:** Java 11 HttpClient tutorial

10.4 JSON parsing with Gson

- **Gson User Guide (official):** <https://github.com/google/gson/blob/main/UserGuide.md>. Read the "Object Examples" and "Nested Classes" sections.
- **Gson repo:** <https://github.com/google/gson>
- **Baeldung, Gson tutorials:** <https://www.baeldung.com/gson-deserialization-guide>
- **JSON syntax:** <https://www.json.org/json-en.html>
- Key concepts: `@SerializedName` (for mapping `feels_like` to `feelsLike`), nested object mapping, and `JsonObject` for manual navigation when the structure gets awkward.
- **YouTube search:** Gson tutorial Java JSON parsing

10.5 Maven

- **Maven in 5 Minutes (official):** <https://maven.apache.org/guides/getting-started/maven-in-five-minutes.html>
- **POM reference:** <https://maven.apache.org/pom.html>
- **Maven Central:** <https://central.sonatype.com/>. Where you look up dependency coordinates and versions.
- Concepts: `pom.xml`, dependency scope, the build lifecycle (`clean`, `compile`, `test`, `package`, `install`).
- **YouTube search:** Maven tutorial for beginners Java

10.6 Testing: JUnit 5 & Mockito

- **JUnit 5 User Guide (official):** <https://junit.org/junit5/docs/current/user-guide/>
- **Baeldung, JUnit 5 guide:** <https://www.baeldung.com/junit-5>
- **Mockito docs:** <https://site.mockito.org/>
- **Baeldung, Mockito series:** <https://www.baeldung.com/mockito-series>
- **YouTube:** Java Brains has a JUnit 5 Basics playlist that's widely recommended as a free alternative to the standard books, and it's more than enough to get you running. Search `Java Brains JUnit 5 Basics`, or `JUnit 5 Full Course for Beginners`.
- Concepts: `@Test`, `@BeforeEach`, `assertEquals` / `assertThrows`, `mock()`, `when().thenReturn()`, `verify()`.

10.7 The API itself

- **OpenWeather API hub:** <https://openweathermap.org/api>
- **Current weather docs:** <https://docs.openweather.co.uk/current>
- **Getting started / API key guide:** <https://docs.openweather.co.uk/appid>
- **Practical walkthrough:** <https://dev.to/joycefosterr/how-to-use-the-openweathermap-api-a-practical-guide-57i2>

10.8 Git & GitHub

- **Pro Git book (free):** <https://git-scm.com/book/en/v2>. Chapters 1 through 3 cover everything you need.
- **GitHub Docs:** <https://docs.github.com/en/get-started>
- **Conventional Commits:** <https://www.conventionalcommits.org/>
- **YouTube search:** `Git and GitHub for beginners crash course`

10.9 JavaFX (only for stretch goal 6)

- **OpenJFX getting started:** <https://openjfx.io/openjfx-docs/>
- **Scene Builder:** <https://gluonhq.com/products/scene-builder/>. A drag-and-drop UI designer.
- **YouTube search:** `JavaFX tutorial for beginners`

10.10 Suggested study order

Week	Focus	Deliverable
1	10.2 + 10.3, Phase 0-2	Raw JSON prints from Java
2	10.4 + 10.5, Phase 3-4	Layered app, models populated
3	Phase 5-6	Full console app, bulletproof errors
4	10.6, Phase 7-8	Tested, documented, tagged v1.0

11. Definition of done for v1.0

- ☐ Runs from a clean clone with only a README and an API key
- ☐ No hardcoded secrets anywhere in code or history
- ☐ Can't be crashed by any user input
- ☐ Works gracefully with no internet connection
- ☐ `mvn test` passes offline
- ☐ 70%+ coverage on service and mapper layers
- ☐ Every public method has Javadoc
- ☐ README has setup instructions someone else could actually follow
- ☐ Tagged `v1.0.0` on GitHub

Once all of that's ticked, this is a portfolio project, not a tutorial exercise. That difference is the whole point.